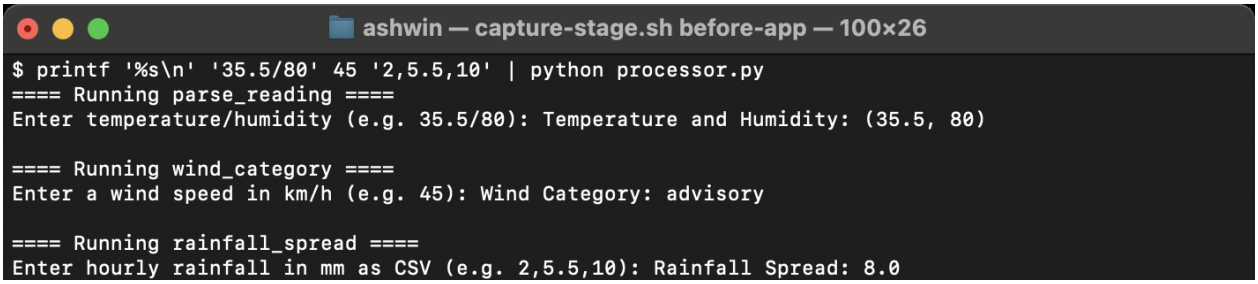
Lab 7: Fuzz Testing

Name - Ashwin Venkatesh

SRN - PES1UG24CS090

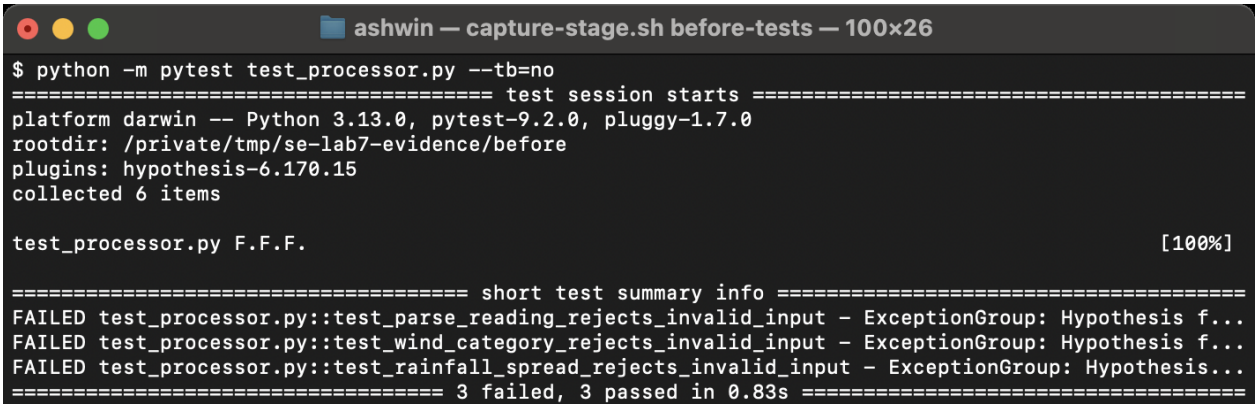
1. Running the starter program

The starter handled the sample readings as expected. It parsed 35.5/80, classified 45 km/h as an advisory and calculated a rainfall spread of 8.0 mm.



2. What the fuzz tests found

For each function, one Hypothesis test checked valid inputs and another checked invalid ones. The three valid-input tests passed. Each invalid-input test found at least one case where the program raised an exception or accepted a bad reading.

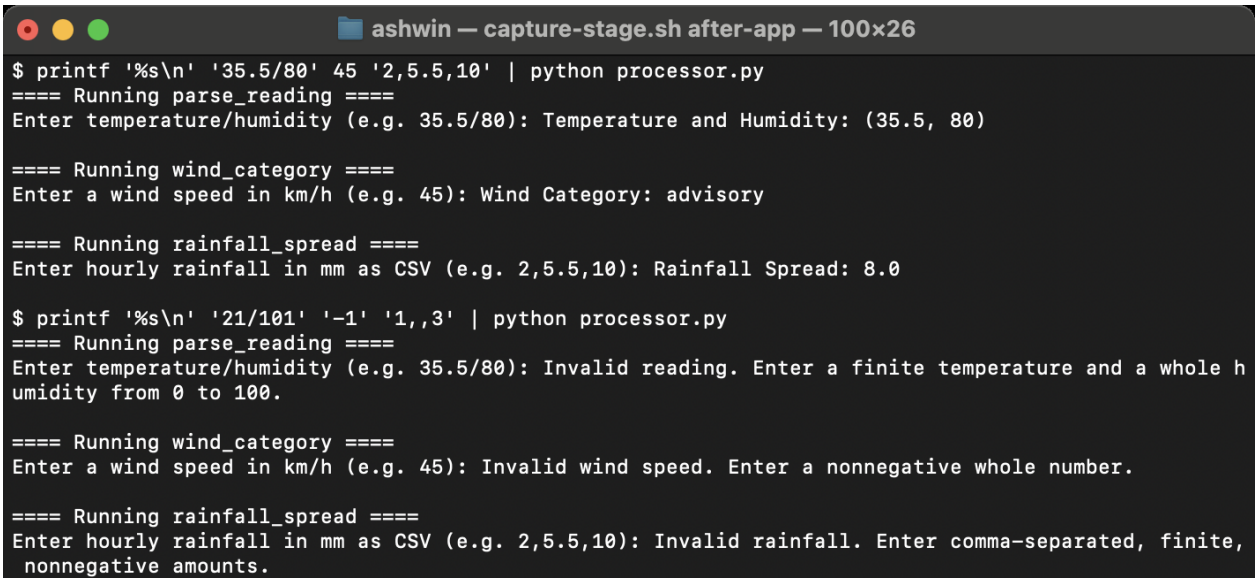


Function	Input	What the starter returned
parse_reading	"nan/50"	A reading with NaN as the temperature.
wind_category	False	The category "none" because False was treated as zero.
rainfall_spread	"0,nan,3"	A spread of 3.0 with a non-finite reading in the list.

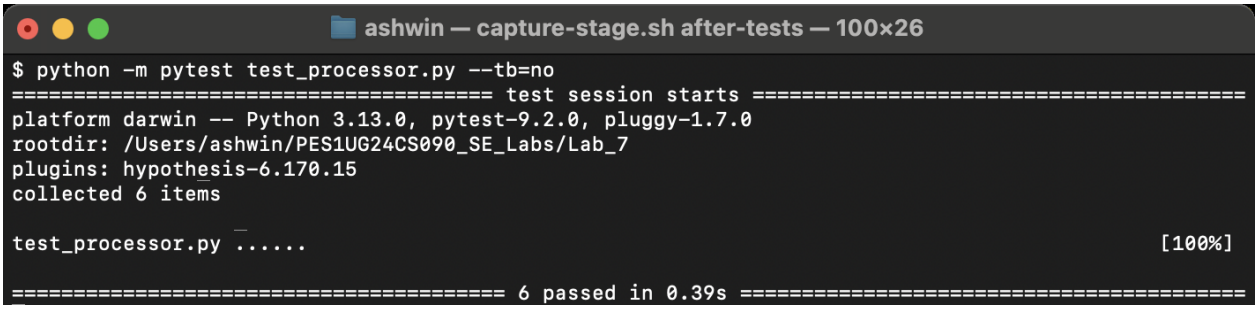
3. Fixing the input checks

The three functions now return None for invalid input. A weather reading needs a finite temperature and a whole humidity percentage from 0 to 100. A wind speed must be a nonnegative whole number. Each rainfall value must be finite and nonnegative, and the list must contain at least one value.

The fixed program still gives the same results for the sample readings. When given 21/101, -1 and 1,,3, it explains what is wrong with each entry.



4. Running the same tests again



All six tests passed after the changes. The valid-input tests also checked the wind thresholds at 39 and 89 km/h and verified that reversing the rainfall readings gives the same spread.

Commands to rerun the tests from Lab_7

```
python -m pip install -r requirements.txt
python -m pytest test_processor.py
```

processor.py

```
"""Parse weather readings and return None for invalid inputs."""

import math


def parse_reading(text):
    """Parse a finite Celsius temperature and a whole humidity percentage."""
    if not isinstance(text, str):
        return None
    try:
        temperature_text, humidity_text = text.split("/")
        temperature = float(temperature_text)
        humidity = int(humidity_text)
    except ValueError:
        return None
    if not math.isfinite(temperature) or not 0 <= humidity <= 100:
        return None
    return temperature, humidity


def wind_category(speed_text):
    """Classify a nonnegative whole-number wind speed in km/h."""
    if not isinstance(speed_text, str):
        return None
    try:
        speed = int(speed_text)
    except ValueError:
        return None
    if speed < 0:
        return None
    if speed >= 89:
        return "warning"
    if speed >= 39:
        return "advisory"
    return "none"
```

processor.py (continued)

```
def rainfall_spread(readings_csv):
    """Return the range of one or more finite, nonnegative rainfall readings."""
    if not isinstance(readings_csv, str):
        return None
    try:
        readings = [float(field) for field in readings_csv.split(",")]
    except ValueError:
        return None
    if any(not math.isfinite(reading) or reading < 0 for reading in readings):
        return None
    return max(readings) - min(readings)


def main():
    print("==== Running parse_reading ====")
    text = input("Enter temperature/humidity (e.g. 35.5/80): ")
    reading = parse_reading(text)
    if reading is None:
        print("Invalid reading. Enter a finite temperature and a whole humidity from 0 to 100.")
    else:
        print("Temperature and Humidity:", reading)

    print("\n==== Running wind_category ====")
    text = input("Enter a wind speed in km/h (e.g. 45): ")
    category = wind_category(text)
    if category is None:
        print("Invalid wind speed. Enter a nonnegative whole number.")
    else:
        print("Wind Category:", category)

    print("\n==== Running rainfall_spread ====")
    text = input("Enter hourly rainfall in mm as CSV (e.g. 2,5.5,10): ")
    spread = rainfall_spread(text)
    if spread is None:
        print("Invalid rainfall. Enter comma-separated, finite, nonnegative amounts.")
    else:
        print("Rainfall Spread:", spread)


if __name__ == "__main__":
    try:
        main()
    except EOFError:
        print("\nInput ended before all weather readings were entered.")
```

test_processor.py

```
import math

from hypothesis import example, given, settings, strategies as st

from processor import parse_reading, rainfall_spread, wind_category


NON_TEXT = st.one_of(
    st.none(), st.booleans(), st.integers(), st.floats(), st.binary(),
    st.lists(st.integers(), max_size=5), st.dictionaries(st.text(), st.integers(), max_size=5),
)
JUNK_TEXT = st.text(alphabet="abcXYZ!?_", min_size=1, max_size=20)
TEMPERATURES = st.floats(allow_nan=False, allow_infinity=False)
HUMIDITIES = st.integers(min_value=0, max_value=100)
WIND_SPEEDS = st.integers(min_value=0, max_value=10**9)
RAINFALL = st.floats(min_value=0, allow_nan=False, allow_infinity=False)
NONFINITE = st.sampled_from(["nan", "inf", "-inf", "1e309"])


@given(st.one_of(
    NON_TEXT,
    JUNK_TEXT,
    st.sampled_from(["", " ", "/", "21/", "/50", "21/50/40", "21/50.5"]),
    NONFINITE.map(lambda temperature: f"{temperature}/50"),
    st.integers(min_value=101).map(lambda humidity: f"21/{humidity}"),
    st.integers(max_value=-1).map(lambda humidity: f"21/{humidity}"),
))
@settings(max_examples=200, derandomize=True)
def test_parse_reading_rejects_invalid_input(reading):
    assert parse_reading(reading) is None


@given(TEMPERATURES, HUMIDITIES)
@example(-12.5, 0)
@example(35.5, 100)
@settings(derandomize=True)
def test_parse_reading_round_trips_valid_input(temperature, humidity):
    result = parse_reading(f" {temperature!r} / {humidity} ")
    assert result == (temperature, humidity)
    assert type(result[0]) is float
    assert type(result[1]) is int
```

test_processor.py (continued)

```
@given(st.one_of(
    NON_TEXT,
    JUNK_TEXT,
    NONFINITE,
    st.sampled_from(["", " ", "45.0", "4e1", "39 km/h"]),
    st.integers(max_value=-1).map(str),
))
@settings(max_examples=200, derandomize=True)
def test_wind_category_rejects_invalid_input(speed_text):
    assert wind_category(speed_text) is None


@given(WIND_SPEEDS)
@example(0)
@example(38)
@example(39)
@example(88)
@example(89)
@settings(derandomize=True)
def test_wind_category_preserves_thresholds(speed):
    expected = {0: "none", 1: "advisory", 2: "warning"}
    assert wind_category(f" {speed} ") == expected[(speed >= 39) + (speed >= 89)]


@given(st.one_of(
    NON_TEXT,
    JUNK_TEXT,
    st.sampled_from(["", " ", ",", ",1", "1,", "1,,2", "1, ,2", "1,heavy"]),
    NONFINITE.map(lambda reading: f"0,{reading},3"),
    st.integers(max_value=-1).map(lambda reading: f"0,{reading},3"),
))
@settings(max_examples=200, derandomize=True)
def test_rainfall_spread_rejects_invalid_input(readings_csv):
    assert rainfall_spread(readings_csv) is None


@given(st.lists(RAINFALL, min_size=1, max_size=20))
@example([0.0])
@example([2.0, 5.5, 10.0])
@settings(derandomize=True)
def test_rainfall_spread_is_the_range_in_any_order(readings):
    fields = [f" {reading!r} " for reading in readings]
    spread = rainfall_spread(",".join(fields))
    assert math.isfinite(spread)
    assert spread == max(readings) - min(readings)
    assert rainfall_spread(",".join(reversed(fields))) == spread
```